



Regresión Logística Multiclase

Aprendizaje Supervisado · OvR vs Softmax · Triage Hospitalario en Ecuador

Inteligencia Artificial · Sexto Semestre · Ingeniería de Sistemas de Información · PUCE

Objetivos de Aprendizaje

1 Explicar qué cambia al pasar de clasificación binaria a multiclase (≥ 3 clases) y por qué scikit-learn ya no usa el parámetro `multi_class`.

2 Entrenar y comparar un modelo OvR (OneVsRestClassifier) y uno Softmax (comportamiento por defecto de LogisticRegression) sobre el mismo dataset.

3 Interpretar la matriz de confusión $K \times K$ y elegir entre macro avg y weighted avg según el contexto del problema.

4 Reentrenar el modelo final con el 100% de los datos, exportarlo con joblib y usarlo para predecir un caso nuevo.

De Clasificación Binaria a Multiclase

Extensión natural del problema de clasificación

Definición: la clasificación multiclase es una tarea en la que la variable objetivo y puede tomar tres o más clases mutuamente excluyentes. Cada observación pertenece exactamente a una clase.

Binaria (2 clases)

- $y \in \{0, 1\}$
- Una función sigmoide $\sigma(z)$
- Un único umbral $\tau = 0.5$
- Métricas directas: Precision, Recall, F1

Multiclase (≥ 3 clases)

- $y \in \{0, 1, \dots, K-1\}$
- Una función Softmax con K salidas
- K probabilidades que siempre suman 1.0
- Métricas: macro avg y weighted avg

Clases mutuamente excluyentes: cada observación pertenece a UNA sola clase. Ejemplo real de este curso: el riesgo de un paciente en urgencias (Bajo, Medio o Alto — nunca dos a la vez). Distinto de un problema multi-etiqueta, donde un mismo elemento sí puede pertenecer a varias categorías a la vez (ej. un paciente con diabetes E hipertensión simultáneamente) — ese caso NO se resuelve con lo que veremos hoy.

Dataset — Triage Hospitalario en Ecuador

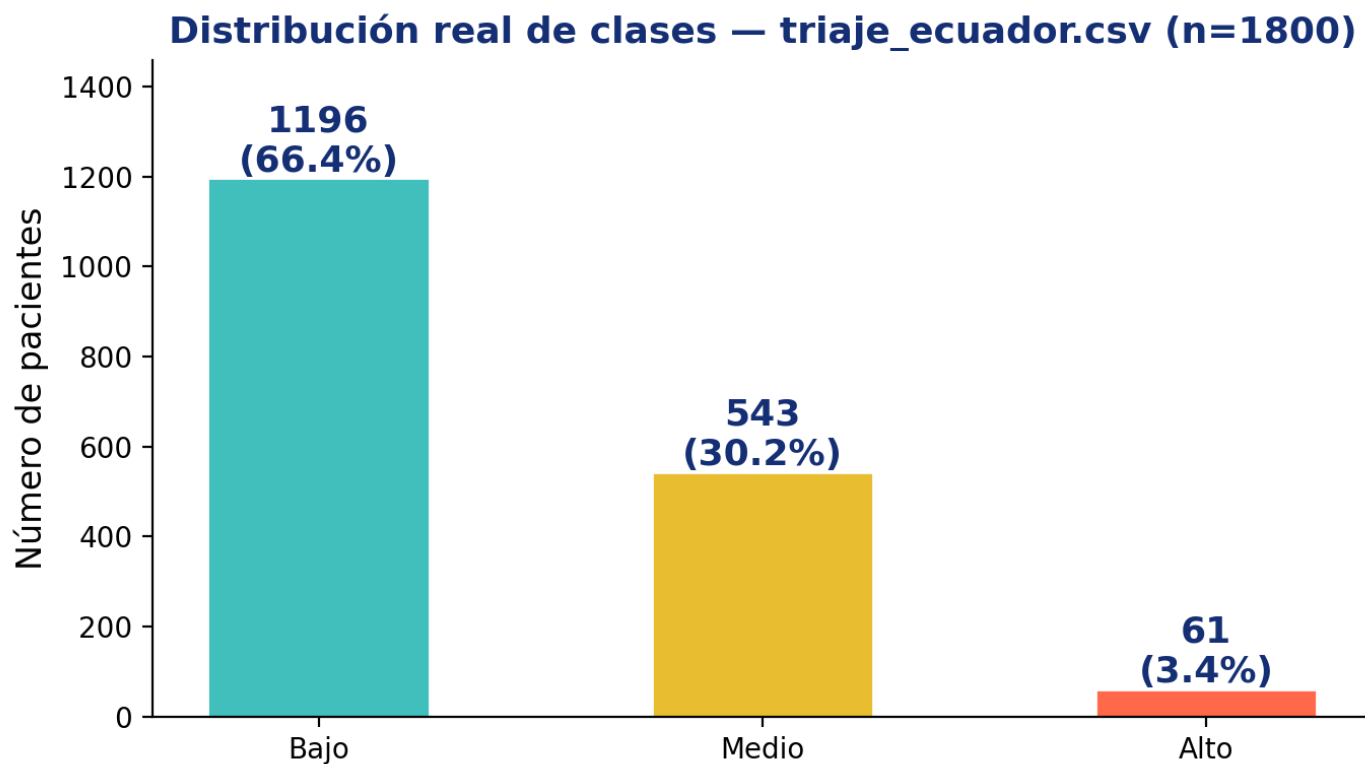
Hospital IESS · Dataset sintético basado en parámetros clínicos reales

Contexto: predecir el nivel de riesgo de un paciente al ingresar a urgencias, para priorizar la atención médica. Variable objetivo: riesgo → Bajo / Medio / Alto.

Variable	Tipo	Descripción
edad	Numérica	Edad del paciente en años
frecuencia_cardiaca	Numérica	Latidos por minuto
presion_sistolica	Numérica	Presión arterial sistólica (mmHg)
temperatura	Numérica	Temperatura corporal en °C
saturacion_o2	Numérica	Saturación de oxígeno (%)
sexo	Categórica	M / F
provincia	Categórica	Provincia de residencia (10 provincias)
motivo_consulta	Categórica	Dolor / Trauma / Respiratorio / Otro
riesgo	Objetivo (y)	Bajo / Medio / Alto

Dataset — Distribución Real de Clases

triaje_ecuador.csv · n = 1800 pacientes



Desbalance real: Riesgo Bajo domina el dataset (66.4%) y Riesgo Alto es apenas 3.4% (61 de 1800 pacientes). Este desbalance es la razón por la que Accuracy sola no basta para evaluar el modelo.

Consecuencia práctica

Un modelo que siempre prediga 'Bajo' tendría Accuracy $\approx 66\%$, pero sería clínicamente inútil: nunca detectaría un caso de Riesgo Alto.

Preprocesamiento en Clasificación Multiclase

¿Qué cambia respecto al caso binario?

Buenas noticias: el preprocesamiento de las variables independientes (X) es idéntico al caso binario — OHE para categóricas y StandardScaler para numéricas. La única diferencia está en cómo se trata la variable objetivo y.

Variables independientes X — igual que binaria

- OneHotEncoder para las variables categóricas.
- StandardScaler para las variables numéricas.
- No fuga de datos: se ajusta solo con X_train.

Variable objetivo y — no necesita OHE

- scikit-learn acepta y con varias clases, sin transformar.
- y puede ser texto o números — ambos funcionan igual.
- LabelEncoder no es necesario en este caso.

La Clase LogisticRegression() en Multiclase (1/2)

```
from sklearn.linear_model import LogisticRegression

# Con 3 o más clases en y, LogisticRegression() usa Softmax
# (multinomial) automáticamente. No requiere ningún parámetro extra.

modelo = LogisticRegression(
    solver='lbfgs', max_iter=1000, random_state=42
)
```

En las siguientes diapositivas: se explican las dos estrategias posibles para multiclase, y cómo obtener cada una con scikit-learn.

Dos Estrategias para Extender la Regresión Logística a K Clases

¿Cómo pasa un clasificador binario a decidir entre 3 o más clases?

El problema: la regresión logística binaria produce un solo número — la probabilidad de la clase positiva, con una sola sigmoide. Con K clases no hay una única clase "positiva": existen dos estrategias distintas, ambas construidas sobre `LogisticRegression()`, para resolver esto.

One-vs-Rest (OvR)

- Entrena K clasificadores binarios independientes.
- Cada uno responde una pregunta distinta: "¿es esta clase, o es cualquiera de las otras?"
- Se predice la clase cuyo clasificador dio la probabilidad más alta.

Softmax (multinomial)

- Entrena un único modelo con K salidas simultáneas.
- Generaliza directamente la fórmula sigmoide a K clases (siguiente diapositiva).
- Las K probabilidades siempre suman exactamente 1.0.

Ambas parten del mismo modelo base, `LogisticRegression()`: Softmax es su comportamiento nativo (ya visto). OvR se obtiene envolviéndola con `OneVsRestClassifier()` — no es una clase distinta, sino un meta-estimador que la reutiliza.

Estrategia Softmax — Regresión Logística Multinomial

Generalización directa de la sigmoide a K clases — el comportamiento por defecto de LogisticRegression()

$$\Pr(Y_i = k \mid X_i) = \frac{e^{\beta_k \cdot X_i}}{\sum_{j=1}^K e^{\beta_j \cdot X_i}}, \quad 1 \leq k \leq K$$

Suma = 1.0

Las probabilidades de las K clases siempre suman exactamente 1.0 y son interpretables directamente.

Clase dominante

La clase predicha es siempre la de mayor probabilidad: $\operatorname{argmax} P(y=k \mid X)$.

K combinaciones

El modelo aprende K conjuntos de coeficientes β — uno por clase; los parámetros crecen con K.

Conexión con la sigmoide

Para K=2, Softmax equivale matemáticamente a la regresión logística binaria con sigmoide.

La Clase LogisticRegression() en Multiclase (2/2)

Parámetros del constructor y el atributo classes_

Parámetros del constructor (se pasan al crear el modelo)

- **solver='lbfgs' (por defecto)** — mismo algoritmo de optimización que en binaria.
- **max_iter=100 por defecto** — se sube a 1000 en este curso como margen de seguridad.
- **random_state=42** — fija la semilla para resultados reproducibles.

classes_ — **no es un parámetro, es un ATRIBUTO:** solo existe DESPUÉS de llamar **fit()**. Guarda el **orden alfabético** en que el modelo indexó las clases internamente. Es la clave para interpretar **predict_proba()**: la columna **i** de esa salida corresponde siempre a **classes_[i]**.

```
modelo.fit(X_train, y_train)
print(modelo.classes_)
# ej.: ['Alto' 'Bajo' 'Medio'] – orden alfabético, no el orden del CSV
```

La Clase OneVsRestClassifier()

Cómo obtener la estrategia OvR con la API actual

Relación con LogisticRegression(): OneVsRestClassifier NO es un modelo alternativo — es un envoltorio (meta-estimador) de sklearn.multiclass que TOMA un clasificador binario (aquí, LogisticRegression) y lo entrena K veces, una por clase, cada vez resolviendo "esta clase vs el resto". Al final usa la probabilidad más alta entre los K para decidir.

```
from sklearn.multiclass import OneVsRestClassifier
from sklearn.linear_model import LogisticRegression

# Envuelve un LogisticRegression binario y lo replica K veces
modelo_ovr = OneVsRestClassifier(
    LogisticRegression(solver='lbfgs', max_iter=1000, random_state=42)
)
```

modelo_ovr.classes_ también existe y sigue el mismo orden que en LogisticRegression().

- Más adelante se combinará con el preprocesamiento dentro de un Pipeline, igual que el modelo Softmax.

OvR vs Softmax — Comparación Conceptual

Criterios prácticos de selección

Criterio	OvR (OneVsRestClassifier)	Softmax (LogisticRegression, por defecto)
¿Cómo funciona?	K clasificadores binarios independientes.	Un único modelo con K salidas simultáneas.
Probabilidades	No garantizan sumar 1.0 entre los K clasificadores.	Siempre suman exactamente 1.0.
Clases minoritarias	Cada clasificador ve 'esta clase' vs 'el resto' — puede ignorar clases raras.	Aprende los K coeficientes a la vez, comparando clases directamente.
scikit-learn	OneVsRestClassifier(LogisticRegression())	LogisticRegression() — comportamiento por defecto.
¿Cuándo usarlo?	Clases no excluyentes, o para paralelizar modelos grandes.	Caso más común: clases mutuamente excluyentes.

Regla práctica: para la mayoría de problemas con clases mutuamente excluyentes, usar Softmax (el valor por defecto de LogisticRegression()). Lo comprobamos con datos reales más adelante.

La Clase ColumnTransformer()

Un solo objeto que aplica OHE a las categóricas y escalado a las numéricas

ColumnTransformer: clase de scikit-learn que aplica una transformación distinta a distintos grupos de columnas de un DataFrame, y devuelve el resultado combinado en una sola matriz.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
cat_cols = ['sexo', 'motivo_consulta', 'provincia']
num_cols = ['edad', 'frecuencia_cardiaca', 'presion_sistolica',
            'temperatura', 'saturacion_o2']
preproc = ColumnTransformer([
    ('cat', OneHotEncoder(drop='first',
                          handle_unknown='ignore'), cat_cols),
    ('num', StandardScaler(), num_cols),
])
```

handle_unknown='ignore' evita errores ante una categoría nueva en producción. preproc por sí solo no entrena nada — se combina con el modelo dentro de un Pipeline.

La Clase Pipeline()

Un solo objeto que encadena preprocesamiento + modelo — igual para las dos estrategias

Pipeline: clase de scikit-learn que encadena preprocesamiento y modelo en un solo objeto: fit()/predict() ejecutan todos los pasos en orden. Se construye igual para ambas estrategias — solo cambia el estimador que se pasa en ('modelo', ...).

```
from sklearn.pipeline import Pipeline  
pipeline_soft = Pipeline([  
    ('preproc', preproc),  
    ('modelo', LogisticRegression(solver='lbfgs',  
                                   max_iter=1000, random_state=42)),  
])  
  
pipeline_ovr = Pipeline([  
    ('preproc', preproc),  
    ('modelo', OneVsRestClassifier(  
        LogisticRegression(solver='lbfgs',  
                            max_iter=1000, random_state=42))),  
])
```

Construyendo X, y y Entrenando los Dos Pipelines (1/2)

Secuencia correcta de pasos — siempre en este orden

1. Preparar X e y

```
X = df.drop('riesgo', axis=1)    # todo menos el objetivo  
y = df['riesgo']               # 'Bajo' / 'Medio' / 'Alto', sin transformar
```

2. Dividir train/test

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y)  
# stratify=y mantiene la misma proporción de las 3 clases  
# en train y test – clave porque Alto es solo 3.4% del total
```

3. Entrenar ambos pipelines

```
pipeline_ovr.fit(X_train, y_train)  
pipeline_soft.fit(X_train, y_train)
```

Prediciendo y Evaluando con Ambos Pipelines (2/2)

Secuencia correcta de pasos — continuación

4. Predecir

```
y_pred_ovr = pipeline_ovr.predict(X_test)
y_pred_soft = pipeline_soft.predict(X_test)
y_proba_soft = pipeline_soft.predict_proba(X_test) # K columnas
```

5. Evaluar

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_soft))
```

Regla crítica:

- **pipeline.fit()** ajusta **preproc** y modelo SOLO con **X_train**.
- **pipeline.predict()/predict_proba()** aplican **preproc.transform()** (nunca fit) antes de predecir con **X_test** — la disciplina train/test queda garantizada automáticamente, sin aplicarla a mano.

Matriz de Confusión Multiclase

Una tabla $K \times K$: cada fila es la clase real, cada columna la predicha

Extensión: en multiclase, la matriz es de $K \times K$. Los valores correctos siguen en la diagonal principal. Los errores están fuera de la diagonal y revelan qué clases confunde el modelo.

	Predicho: Bajo	Predicho: Medio	Predicho: Alto
Real: Bajo	Correcto	Error	Error
Real: Medio	Error	Correcto	Error
Real: Alto	Error	Error	Correcto

Interpretación

No todos los errores son iguales: clasificar un paciente de Riesgo Alto como Bajo es mucho más grave clínicamente que confundir Bajo con Medio.

La matriz permite identificar exactamente dónde se concentran los errores críticos.

Matriz de Confusión — Resultados Reales

triaje_ecuador.csv · test set real, n=360 (stratify=y)

OvR (OneVsRestClassifier)

True label	Bajo	Medio	Alto
Bajo	220	19	0
Medio	63	46	0
Alto	3	9	0
		Predicted label	

Softmax (multinomial, por defecto)

True label	Bajo	Medio	Alto
Bajo	215	24	0
Medio	58	49	2
Alto	2	7	3
		Predicted label	

Lectura clave: de los 12 pacientes reales de Riesgo Alto en el test set, OvR predice correctamente 0 (¡ninguno!) mientras que Softmax predice correctamente 3. Con una clase tan minoritaria (Alto = 3.4% del dataset), la estrategia elegida sí cambia el resultado clínico.

Métricas Multiclase — Macro Average

Extensión de Precision, Recall y F1 para K clases

Definición: el macro average calcula la métrica independientemente por cada clase y luego promedia los K resultados con el mismo peso para todas las clases, **sin importar cuántas observaciones tenga cada una.**

$$\text{Macro F1} = \frac{F1_{\text{clase}_0} + F1_{\text{clase}_1} + \dots + F1_{\text{clase}_{K-1}}}{K}$$

Riesgo Bajo

- F1 = 0.84
- n = 239 casos

Riesgo Medio

- F1 = 0.52
- n = 109 casos

Riesgo Alto

- F1 = 0.35
- n = 12 casos

Macro F1 = (0.84 + 0.52 + 0.35) / 3 = 0.569 → cada clase pesa igual, aunque Alto tenga muchos menos casos.

Métricas Multiclase — Weighted Average

Ponderación por el número de observaciones de cada clase

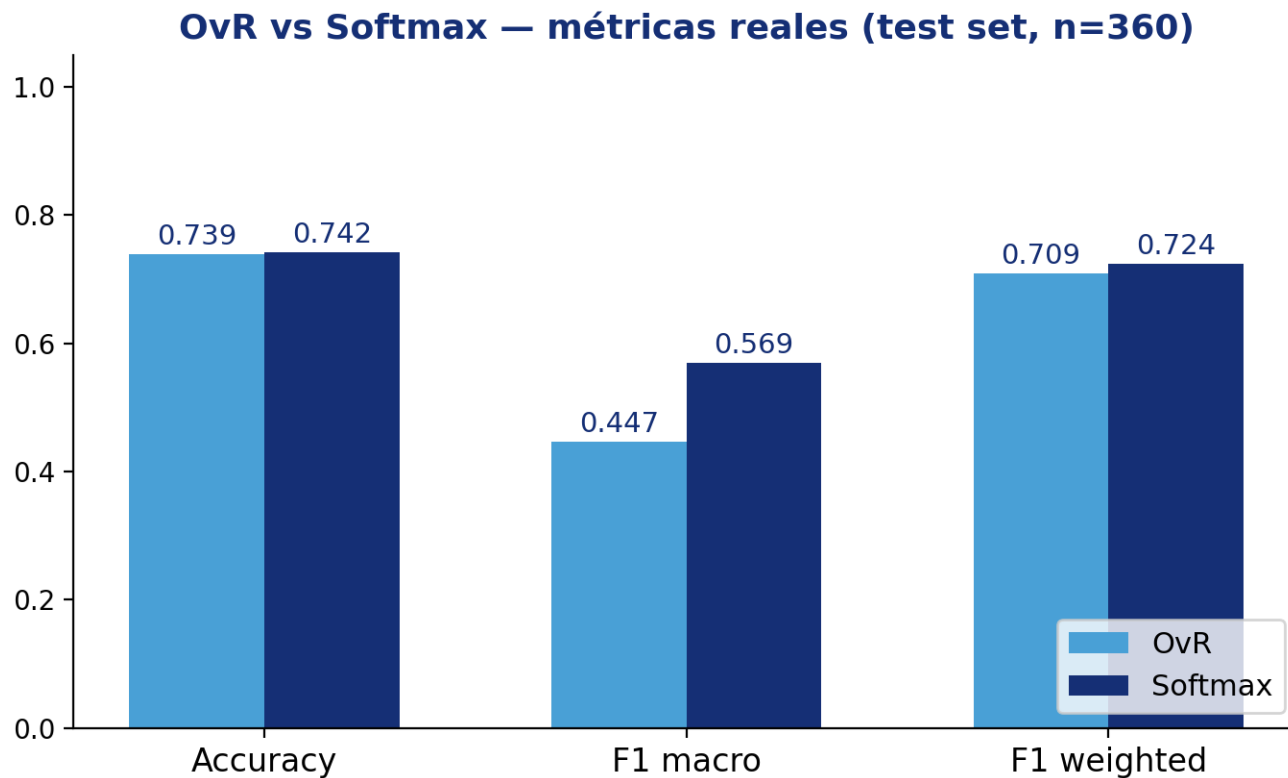
Definición: el weighted average calcula la métrica por clase y promedia ponderando por el número de observaciones (support) de cada clase. Las clases con más casos tienen mayor peso en el promedio final.

$$\text{Weighted F1} = \frac{\sum_k (F1_k \times n_k)}{n_{\text{total}}}$$

Clase	F1	n (support)	Peso = n/360
Riesgo Bajo	0.837	239	0.664
Riesgo Medio	0.519	109	0.303
Riesgo Alto	0.353	12	0.033

Macro F1 = 0.569 ↔ Weighted F1 = 0.724. La diferencia grande muestra el efecto del desbalance: si Riesgo Alto es clínicamente crítico, confiar en macro avg, no en weighted avg ni en Accuracy.

OvR vs Softmax — ¿Cuál Elegir? Resultados Reales



Hallazgo clave: con Riesgo Alto en solo 3.4% del dataset, OvR obtiene recall = 0.000 en esa clase (no detecta NINGÚN caso real de Alto), mientras Softmax sí detecta una parte (recall = 0.25). El Accuracy de ambos es casi idéntico (0.739 vs 0.742) — otra prueba de por qué Accuracy solo no basta con clases desbalanceadas.

En este dataset, con una clase minoritaria clínicamente crítica, Softmax es la mejor elección.

predict_proba en Clasificación Multiclase

El modelo devuelve una probabilidad por cada clase

Extensión: en clasificación binaria, **predict_proba** devuelve 2 columnas [P(y=0), P(y=1)]. **En multiclase devuelve K columnas, una por clase**, en el orden de `modelo.classes_`. Las probabilidades de cada fila siempre suman 1.0.

Variable	Valor
Edad	24 años
Sexo	M
Provincia	Pichincha
Motivo	Respiratorio
Frec. cardíaca	118 lpm
Presión sistólica	98 mmHg
Temperatura	37.2 °C
Saturación O ₂	88 % (crítica, normal ≥95%)

Clase	Probabilidad
Alto	0.073
Bajo	0.059
Medio	0.868

Clase predicha: Medio (P máxima = 0.868)

Nota clínica: aunque el paciente es joven, su saturación de oxígeno crítica (88%) y taquicardia leve elevan su probabilidad de riesgo Medio/Alto muy por encima de lo que sugeriría solo la edad — el modelo captura esa señal.

Entrenamiento Final con el 100% de los Datos

Una vez elegida la estrategia (Softmax), así se prepara el modelo para producción

- El train/test split (80/20) sirvió para MEDIR qué tan bien generaliza el modelo — eso ya se hizo antes.
- Una vez validado ese resultado, el modelo de producción se reentrena con el 100% de los datos.

Modelo de evaluación (80%)

- Se entrena solo con `X_train`.
- Su único propósito es medir precisión, recall, F1 por clase.

Pipeline final (100%)

- Se reentrena con el dataset completo (`X + y`).
- Es el pipeline que se exporta con joblib.

```
pipeline_final = Pipeline([
    ('preproc', preproc),
    ('modelo', LogisticRegression(
        solver='lbfgs', max_iter=1000, random_state=42)),
])
pipeline_final.fit(X, y)    # X, y completos – no X_train
```

pipeline_final aplica preproc y entrena el modelo en una sola llamada, sobre el 100% de los datos.

Exportación del Pipeline con joblib

Empaquetar el pipeline entrenado en un archivo reutilizable

- joblib guarda objetos de Python (como un pipeline entrenado) en un archivo .pkl.
- Al exportar pipeline_final se guardan preproc y modelo juntos, como un solo objeto.

```
import joblib
```

```
joblib.dump(pipeline_final, 'modelo_triaje.pkl')
```

Comparado con guardar el modelo y el preproc por separado

- Un solo objeto, un solo archivo: preproc y modelo nunca pueden desincronizarse ni cargarse en el orden equivocado.

Uso del Modelo con Datos Nuevos

Así se usa el archivo exportado desde otro programa, sin el notebook

```
pipeline_prod = joblib.load('modelo_triage.pkl')

paciente_nuevo = pd.DataFrame({
    'edad': [24], 'sexo': ['M'], 'provincia': ['Pichincha'],
    'motivo_consulta': ['Respiratorio'], 'frecuencia_cardiaca': [118],
    'presion_sistolica': [98], 'temperatura': [37.2],
    'saturacion_o2': [88.0]
})

pred = pipeline_prod.predict(paciente_nuevo)
proba = pipeline_prod.predict_proba(paciente_nuevo)
```

Por qué esto es tan simple

- **pipeline_prod** ya incluye el preprocesamiento: `predict()` y `predict_proba()` aplican preproc por dentro.
- Sigue funcionando con pacientes nuevos gracias a **handle_unknown='ignore'**.